

# Web-Based Citizen Complaint & Service Request Management System (Adama City Administration)

Based on

Project\_Proposal\_Web\_Based\_Citizen\_Complaint\_and\_Service\_Request.pdf.

## Tech Stack (proposal Section 11)

- **Frontend:** React.js (Vite) + React Router + Axios, clean modern responsive CSS
- **Backend:** Node.js + Express.js, Mongoose (MongoDB), JWT authentication, bcrypt password hashing
- **Database:** MongoDB (local)
- **Tools:** VS Code, Postman, Git/GitHub

Out of scope (Section 5): online payments, GIS/maps. Future enhancements (Section 16, not in v1): email/SMS notifications, mobile app, multi-language. V1 satisfies the "notification functionality" requirement with in-app notifications, and "report generation" with a reports module.

## Project Structure

```
plan/
├── server/                                # Express API
│   ├── src/
│   │   ├── models/                      # User, Department, Complaint,
│   │   │   ServiceRequest, Notification
│   │   ├── routes/                      # auth, complaints,
│   │   │   serviceRequests, users, departments, reports, notifications
│   │   ├── middleware/                  # JWT auth, role-based access
│   │   └── control
│   │       └── seed.js                  # admin user, departments,
│   └── sample data
│       └── .env                         # PORT, MONGO_URI, JWT_SECRET
```

```

└─ client/                                # React app
  └─ src/
    └─ pages/                             # citizen, officer, admin pages
    └─ components/                       # navbar, tables, forms, status
badges, notification bell
    └─ api/                              # axios client with auth token

```

## Actors & Workflow (Sections 7 and 10)

```

flowchart LR
    Citizen -->|register and login| System
    Citizen -->|submit| Item[Complaint or Service Request]
    Item -->|stored| DB[(MongoDB)]
    Admin -->|reviews and assigns| Officer[Department Officer]
    Officer -->|"processes, updates status (Pending, In Progress, Resolved)"| Item
    Item -->|status update + in-app notification| Citizen
    Admin -->|generates| Reports[Reports and Statistics]

```

- **Citizen:** register, login, submit complaints and service requests, view status, update profile
- **Administrator:** manage users, manage complaints/requests, assign to departments, update statuses, generate reports, monitor system via dashboard
- **Department Officer:** view assigned complaints/requests, process them, update progress, mark resolved

## Database Design (Section 12)

- **User:** fullName, email, password (bcrypt hash), role (citizen | officer | admin), phoneNumber, department (officers only)
- **Department:** name, description (Road Maintenance, Waste Management, Water Supply, Street Lighting, General Services)
- **Complaint:** title, description, category, status (pending | in\_progress | resolved), citizen (ref), assignedDepartment (ref), assignedOfficer (ref), statusHistory[] (status, note, by, at), createdAt
- **ServiceRequest:** serviceType, description, status, citizen, assignedDepartment, assignedOfficer, statusHistory[], createdAt

- **Notification:** user (ref), message, read, link, createdAt

## API Endpoints (Functional Requirements, Section 8)

- **Auth:** POST /api/auth/register, POST /api/auth/login, PATCH /api/auth/password
- **Complaints:** POST /api/complaints (**citizen**), GET /api/complaints (role-filtered with ?search=&status=&category=&department= and pagination), GET /api/complaints/:id, PATCH /api/complaints/:id/assign (**admin**), PATCH /api/complaints/:id/status (**officer/admin** — appends statusHistory, creates citizen notification)
- **Service Requests:** same set under /api/service-requests
- **Users:** GET/POST/PATCH/DELETE /api/users (**admin:** create officers, manage roles), PATCH /api/users/me (**profile**)
- **Departments:** CRUD /api/departments (**admin**)
- **Reports:** GET /api/reports/summary (counts by status/category/department, monthly trend) — powers admin dashboard and printable reports
- **Notifications:** GET /api/notifications, PATCH /api/notifications/:id/read

## Frontend Pages

- **Public:** Landing page, Register, Login
- **Citizen:** Dashboard (my complaints and service requests with statuses), Submit Complaint, Submit Service Request, detail view with status timeline, Profile (edit info, change password), notification bell
- **Department Officer:** Assigned queue (filterable complaints + requests), detail page with status update form (Pending → In Progress → Resolved) and progress notes
- **Admin:** Dashboard (statistic cards + recent activity), All Complaints / All Service Requests (search, filter, assign to

department/officer), User Management, Department Management, Reports page (filter by date/status/department, printable)

## **Non-Functional Requirements (Section 9)**

- Security: bcrypt hashing, JWT-protected routes, role-based access middleware
- Performance: indexed queries, pagination on lists
- Usability: simple navigation, responsive layout, clear status colors

## **Deliverables**

- Seed script: admin account, departments, demo citizen/officer accounts, sample complaints and requests
- `README.md`: setup steps (MongoDB, env vars, install and run commands for server and client)
- Verified end-to-end workflow matching proposal Section 10